AI + Google + AI Agent +

Google’s OKF, Explained and Put to Work



Saurabh Follow 10 min read · Oct 2, 2026

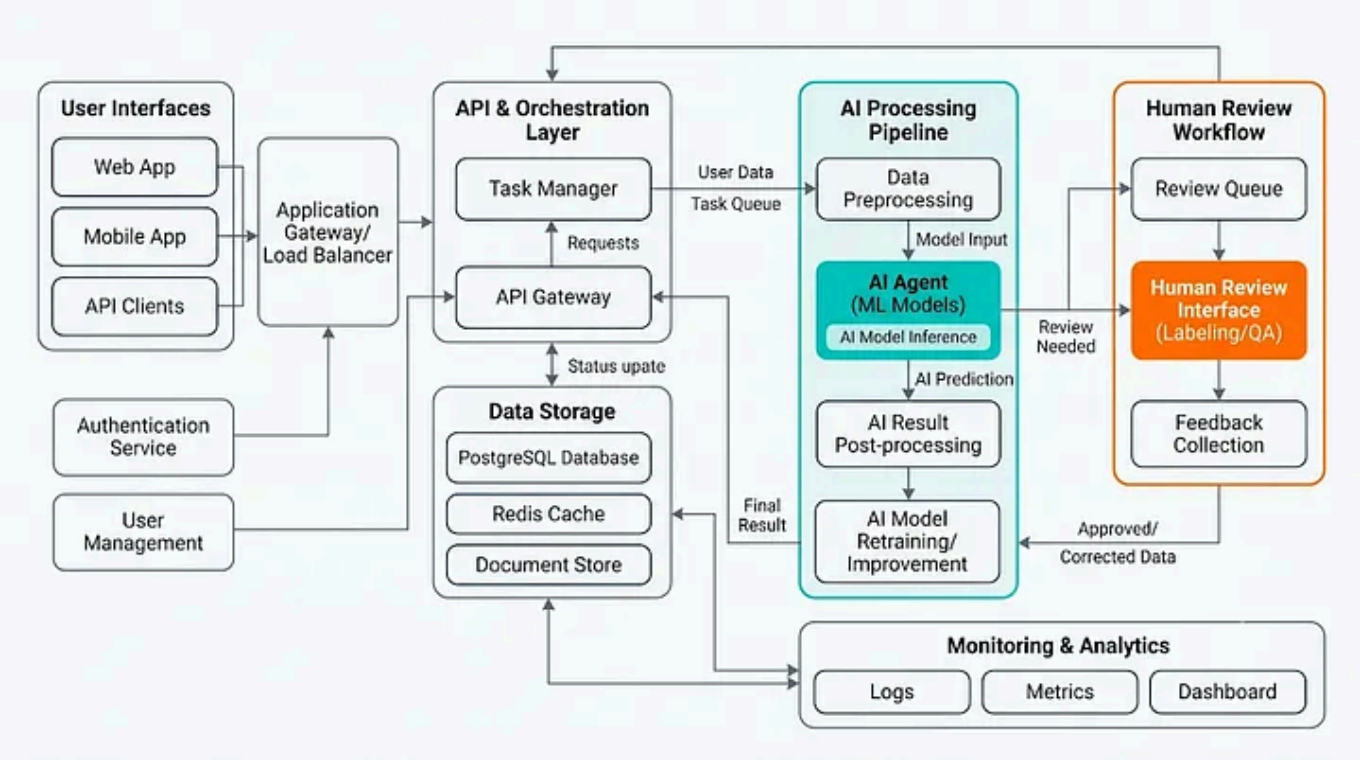
1

Every AI agent you run starts each task with amnesia. It re-reads your files, re-guesses what a table means and re-discovers who owns what, and you pay for that in tokens, time and wrong answers. In June 2026, Google Cloud published a very small standard that attacks exactly this: the Open Knowledge Format, or OKF.

This article explains what OKF is in plain words, why it matters now, how to build a bundle that updates itself, and where it can go beyond code. I’ve separated what the sources confirm from what I couldn’t verify.

What you’ll walk away with

1. **OKF in 30 seconds**, with a real example file.
2. **Why agents need it**, and where RAG and a giant `AGENTS.md` fall short.
3. **What changed in v0.2:** trust, provenance and staleness fields.
4. **Where to use it** beyond a codebase.
5. **A pipeline** that keeps a bundle current on every commit.
6. **The honest limits**, including the criticisms.

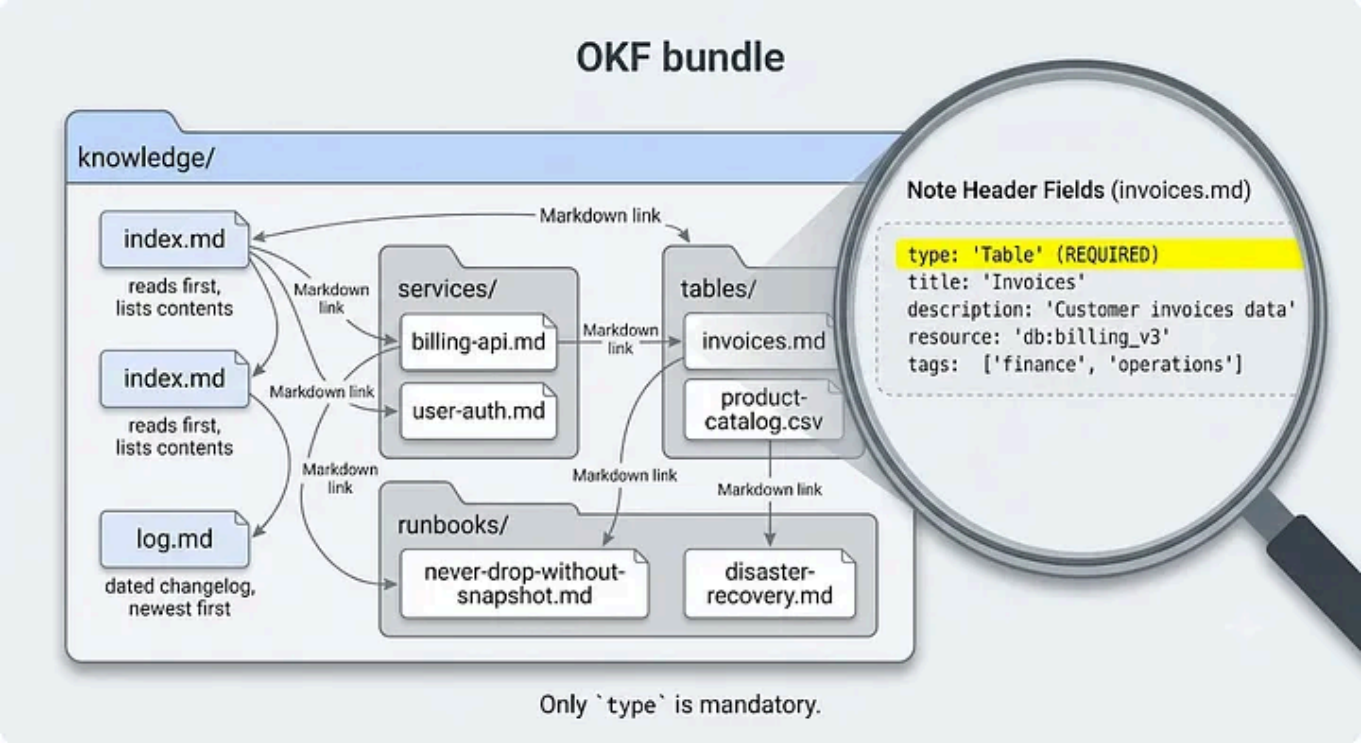


OKF in 30 seconds

OKF is a folder of Markdown notes. Each note describes one thing, such as a table, a service or a runbook. Each starts with a small YAML header, and notes point to each other with ordinary Markdown links. Google released v0.1 on June 12, 2026 under Apache 2.0, from its Data Cloud team (Sam McVeety and Amir Hormati).

The rules are short:

- **One required field:** `type`, the kind of thing the note describes.
- **Recommended fields:** `title`, `description`, `resource` (a link to the real asset) and `tags`.
- **Two reserved files:** `index.md` lists a folder's contents so an agent can read step by step, and `log.md` is a dated changelog, newest first.
- **Links are untyped.** The sentence around a link says how two notes relate.
- **Readers must be forgiving.** A broken link isn’t an error. It can mean knowledge nobody has written yet.



Here is an illustrative note for a codebase service. It is my example, not one from Google’s repo:

```
---
type: Service
title: Billing API
description: Creates invoices and charges customers.
resource: https://github.com/acme/billing
tags: [payments, backend]
generated:
  by: process:post-commit-hook
```

```
  at: 2026-10-02T09:00:00Z
status: stable
stale_after: 2026-11-02T00:00:00Z
---
```

```
# What it does

Charges are created by [the payments worker](/services/payments-
worker.md)
and stored in [the invoices table](/tables/invoices.md).

# Owner

Payments team. Do not change the schema without them.
```

An agent reads `index.md`, picks this note, follows two links and has the context. It never opens the other thousand files.

Why agents need it now

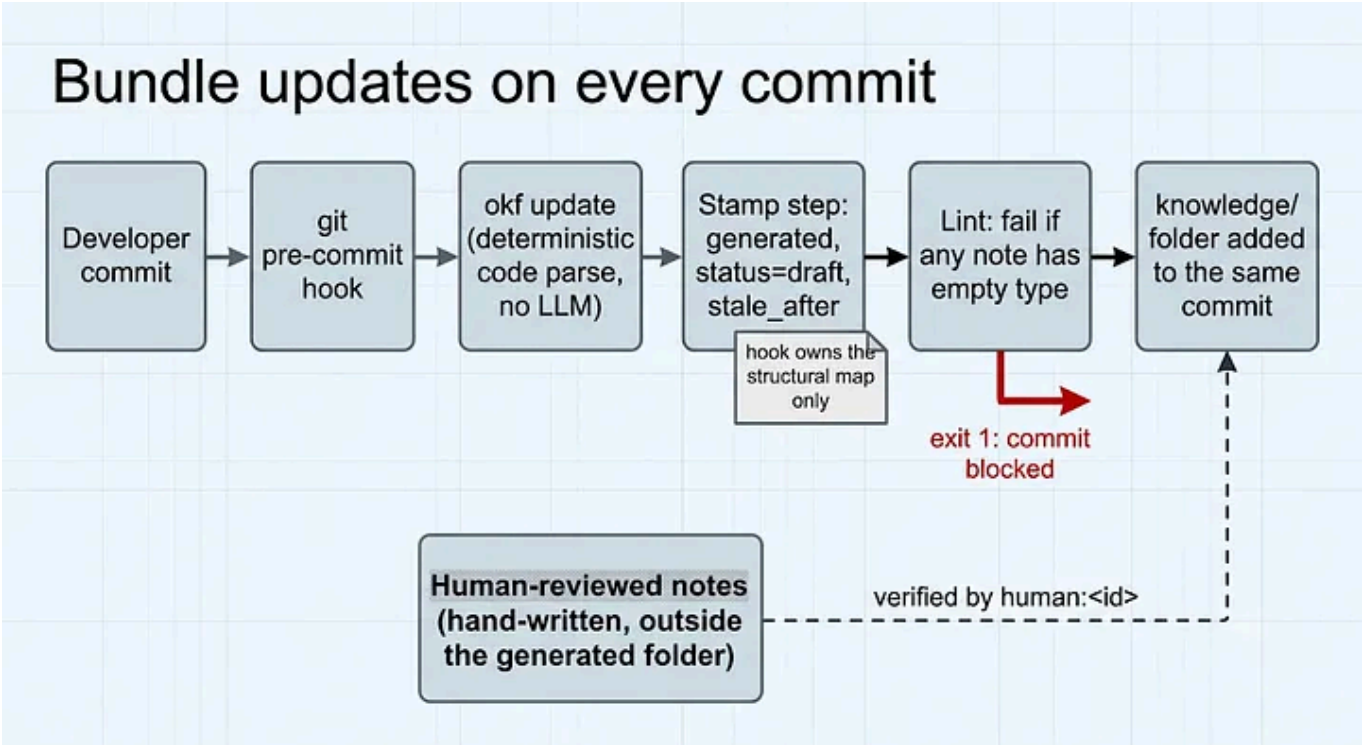
Google's own framing is that organisational knowledge is scattered: schemas in catalogs, metrics in wikis, rules in code comments, and the rest in engineers' heads. Every agent builder then solves the same context-assembly problem alone. OKF is Google's attempt to give that knowledge one portable shape, formalizing what it calls the LLM-wiki pattern.

Approach	How it finds context	Where it breaks
Dump the repo into the prompt	Reads everything	Token cost, and the model still has to work out the structure
RAG (vector search)	Nearest-neighbour match on text chunks	Chunking can split the structure that makes an answer reliable; the match is closest, not necessarily correct
`AGENTS.md` / `CLAUDE.md`	One file of instructions	Tells agents how to behave, not what exists
OKF bundle	Path lookup, then explicit link traversal	Someone has to write and maintain the notes

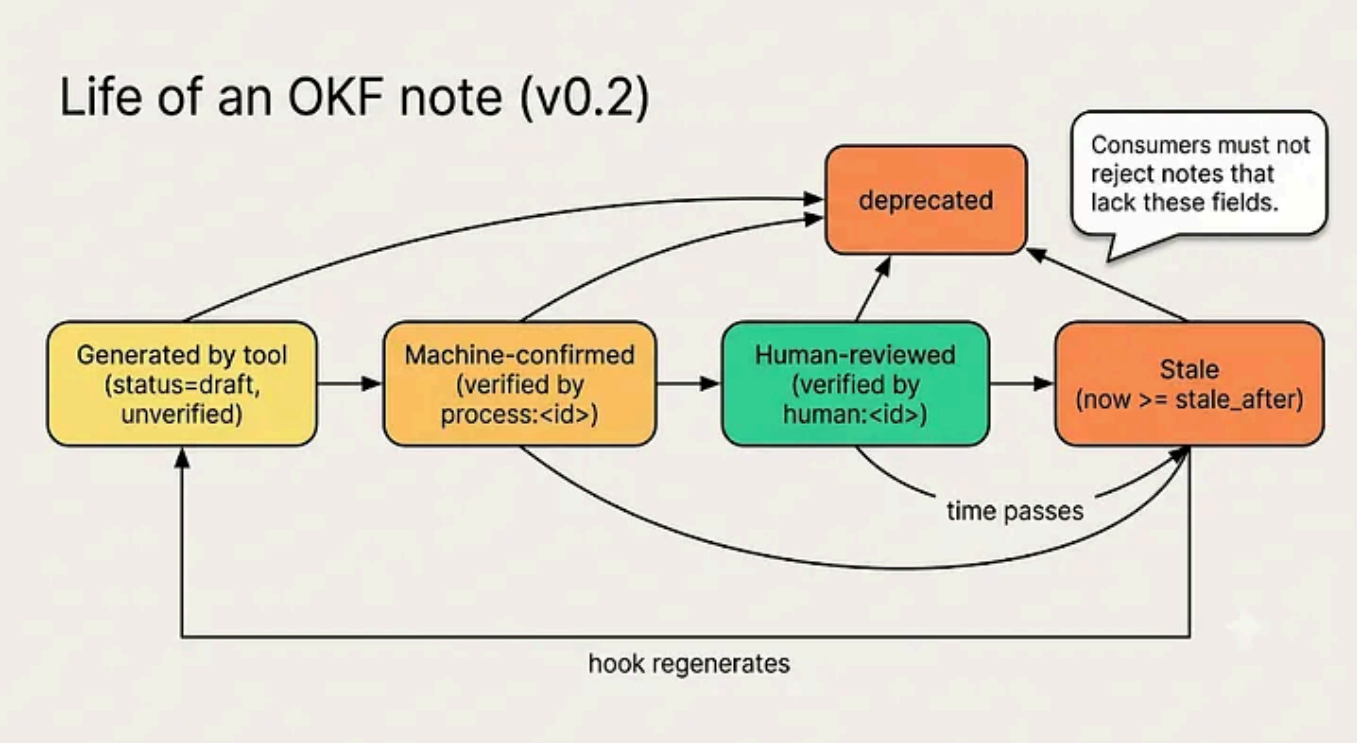
Two points from the sources are worth keeping straight:

- **OKF and AGENTS.md do different jobs.** `AGENTS.md` says how an agent should behave. OKF describes what exists. The first can point at the second.
- **OKF doesn’t kill RAG.** Several write-ups land on the same split: OKF for stable, known facts, and RAG for large, messy archives such as tickets, transcripts and old decks. One article calls it a demotion of vector search to its proper job, not a retirement.

On cost, I found **no independent benchmark**. The tools I opened that claim big token savings are the authors’ own numbers, so treat any “10x cheaper” line, including headlines that say it, as a hypothesis to test on your repo.



What changed in v0.2: trust and staleness



The v0.1 spec said nothing about whether a note was still true. The current spec, v0.2, adds optional fields for exactly that, which is the part that makes a self-updating bundle safe to rely on:

- **`generated`** records who or what wrote the note (`by`) and when (`at`).
- **`verified`** records who checked it. No entry means unverified, a machine-only entry means machine-confirmed, and a `human:` entry means human-reviewed.

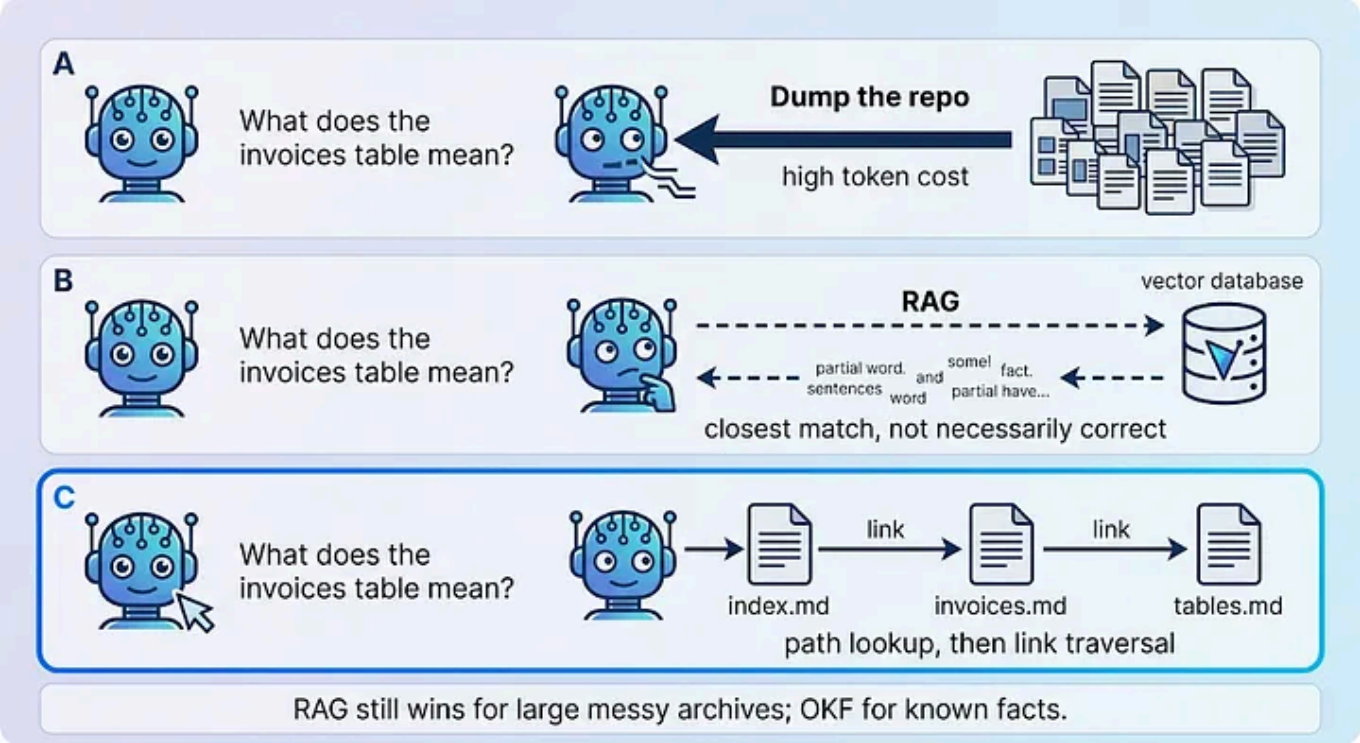
- **`status`** is `draft`, `stable` (the default) or `deprecated`.
- **`stale_after`** is a date after which the note counts as stale.
- **`sources`** lists what the note was derived from, with a `resource` for each entry.
- **Actor names** follow a pattern: `<tool>/<version>`, `human:<id>` or `process:<id>`.

Medium Search Play Games Write

v0.2 also adds a concept type called **Blessed Computation**: a templated query or script that an agent may run by filling in parameters but must never edit, plus a deterministic check that the run happened as approved. It is aimed at metrics such as revenue, where a plausible but different SQL query is the failure you fear.

Two cautions. v0.2 replaces v0.1's `timestamp` with `generated.at`, so older bundles need a small migration. And all of these fields are optional: consumers must not reject a note for lacking them, so nothing forces anyone to fill them in.

Where to use it beyond a codebase



Google's own examples are data catalogs: tables, datasets and metrics. The tools that appeared in the weeks after release show how far the idea stretches. These are the uses I'd actually try, ordered by how well the evidence supports them:

1. **Metric and table definitions.** Google's own samples (GA4 e-commerce, Stack Overflow, Bitcoin datasets) are in this lane. Treat each metric as a reviewed file, so "revenue" means one thing for every agent.

2. **A codebase map.** Services, modules and ownership as linked notes. Several community generators scan a repo and emit a bundle, and some wire it into `CLAUDE.md` or `.cursorrules`.
3. **Runbooks and incident knowledge.** Stable, high-stakes procedures an agent should follow instead of improvise. This fits the agent-guardrails problem directly: a note that says what must never be dropped without a snapshot.
4. **Agent memory.** At least one community tool stores an agent's decisions and observations as OKF notes, so each session builds on the last and a human can read the memory.
5. **Team onboarding and personal knowledge.** Bundles open natively in Obsidian, since they are Markdown with frontmatter, so a vault can double as agent context.
6. **A public site for agents.** One community plugin serves a WordPress site's content as an OKF bundle at `/okf/`.

The pattern across the strong uses is the same: **small, durable, high-value concepts, not a bulk export of everything your company ever wrote.** One practitioner write-up recommends a pilot of 20 to 50 concepts, one owner

group and one measurable workflow, and judging it on answer correctness rather than demo polish. That is good advice for anyone.

Build it: a bundle that updates on every commit

I built and ran a small version of this on a toy two-file repo. It is a test, not a benchmark, and it says nothing about a thousand-file codebase. The pipeline has three steps that run in a `pre-commit` hook:

1. **Generate or update** the bundle from the code. I used the community tool `okf-generator` (v0.1.53), which parses code deterministically with no LLM. It produced 6 notes from 2 Python files in 0.04 seconds, with `calls` and `related` links between functions.
2. **Stamp** every new note with v0.2 trust fields: `generated`, `status: draft` and `stale_after`. The tool wrote an `okf_version: "0.2"` header and a `timestamp`, but none of the trust fields, so I added this step myself.
3. **Lint** the bundle: fail the commit if any note lacks a non-empty `type`, the one rule the spec makes mandatory.

The stamper (about 30 lines of Python with PyYAML):

```
#!/usr/bin/env python3
import sys, re, datetime as dt, pathlib, yaml
```

```
RESERVED = {"index.md", "log.md", "SUMMARY.md"}
FM = re.compile(r"\A---\n(.*?)\n---\n(.*)\Z", re.S)

def main(bundle, days=30):
    now = dt.datetime.now(dt.timezone.utc).replace(microsecond=0)
    iso = lambda d: d.strftime("%Y-%m-%dT%H:%M:%SZ")
    stale, stamped, bad = now + dt.timedelta(days=days), 0, []
    for p in sorted(pathlib.Path(bundle).rglob("*.md")):
        if p.name in RESERVED:
            continue
        m = FM.match(p.read_text())
        if not m:
            bad.append(f"{p}: no frontmatter"); continue
        fm = yaml.safe_load(m.group(1)) or {}
        if not str(fm.get("type") or "").strip():
            bad.append(f"{p}: empty type"); continue
        if "generated" in fm or "verified" in fm:
            continue
        fm["generated"] = {"by": "process:okf-hook", "at": iso(now)}
        fm["status"] = "draft"
        fm["stale_after"] = iso(stale)
        p.write_text("---\n" + yaml.safe_dump(fm,
sort_keys=False).strip()
                     + "\n---\n" + m.group(2))
        stamped += 1
    print(f"stamped {stamped}; {len(bad)} problem(s)", *bad, sep="\n
")
    return 1 if bad else 0

sys.exit(main(sys.argv[1]))
```

And the hook, saved as `.githooks/pre-commit` and enabled with `git config core.hooksPath .githooks`:

```
#!/usr/bin/env bash
set -euo pipefail
cd "$(git rev-parse --show-toplevel)"
if [ -d knowledge ]; then okf update . knowledge; else okf generate . knowledge;
python okf_stamp.py knowledge
git add knowledge
```

What I saw when I ran it:

- Adding a function and committing created its note and included it in the same commit.
- Breaking a note's `type` made the linter exit with code 1. My first version missed this, because an empty YAML value comes back as `None`, which I had turned into the string "None". The negative test caught it.
- **The update rewrote the whole bundle's notes, and my stamper re-marked them all as `draft`.** If a human had marked a generated note `verified`, that mark would be lost. So keep human-reviewed knowledge

(owners, "never drop without a snapshot", what a metric really means) in hand-written notes outside the generated folder, and let the hook own only the structural map.

That last point is the real design lesson. The generator can tell an agent that `drop_orders_table` exists and which functions call it. Nothing in the code says the table is irreplaceable. That meaning has to be written by a person, which is where OKF's `verified` field earns its keep.

The honest limits

OKF is promising and very new. Here is what the critics and my own test turned up:

- **It standardizes structure, not meaning.** Two teams can ship valid bundles where “orders” means different things, and an agent can return a number that is syntactically fine and wrong.
- **Types are unregistered.** `type` is free text, so consumers may need to normalize vocabularies. The context-assembly problem can come back as type mapping.
- **Everything around the file is out of scope.** Permissions, secret redaction, approval workflows, freshness checks and data quality are separate

layers. So are deletion propagation and high-frequency writes.

- **Generated notes can be confidently wrong.** A mistake saved in Markdown gets reused by humans and agents alike. The advice from one critic: derive timestamps from git history instead of letting an LLM guess them, and review AI-written cross-links and join explanations by hand.
- **Untrusted input.** A bundle is text an agent will read as context, so a poisoned PDF or web page converted into notes is a prompt-injection route. A community tool, `okf-guard`, exists to screen documents before conversion. I haven't tested it.
- **Adoption is unproven.** “Open”, “vendor-neutral” and “secure” are design goals until independent producers and consumers show up. The spec moved from v0.1 (June 12, 2026) to v0.2 within weeks, so expect churn.
- **No independent token benchmark.** I found none. Savings claims come from tool authors.
- **My test was tiny.** Two files, one language, one run. It shows the mechanics work, not that they scale.

The common thread is that OKF makes knowledge portable and diffable. It doesn’t make it correct. Correctness still needs owners.

If you only do three things

1. **Pick one domain and write 20 to 50 notes by hand** for the things an agent must never get wrong: a few tables, a few metrics, a few runbooks. Quality beats coverage.
2. **Let a hook own the structural map,** and keep human-reviewed notes in a separate folder so regeneration can’t overwrite a review.
3. **Measure one workflow before and after.** Count wrong answers and tokens on a task you repeat. Believe your numbers over anyone’s headline.

Honest summary

OKF isn’t magic and it isn’t a database. It is a naming convention for context: a folder, a header and some links. That is why it spread fast, with tools appearing within weeks, and also why it can’t tell you whether a note is true.

The useful way to think about it is as a contract between people and agents. A generator draws the map, a human signs the parts that matter, and the `verified`, `status` and `stale_after` fields say which is which. Whether the format becomes a standard is still open. A folder of well-organized Markdown is worth having either way.

Sources

All pages below were opened while writing this article.

- OKF v0.2 specification, GoogleCloudPlatform/knowledge-catalog
- Google Cloud: How the Open Knowledge Format can improve data sharing
- Tech Times: Google Cloud OKF announcement
- OKF Bundle: What is OKF v0.1
- PPC Land: OKF as a lingua franca for agent knowledge
- MarkTechPost: Google Cloud introduces OKF
- OKF tools catalog (community site, okf.md) and FAQ
- Harsha Sridhar: Google’s Open Knowledge Format Is Almost Nothing
- SmartScope: OKF critique and adoption caveats
- Wavect: OKF in practice
- MayhemCode: OKF vs RAG
- Analytics Vidhya: OKF
- alexop.dev: OKF, frontmatter and agent knowledge

- okf-generator on PyPI (I ran v0.1.53 myself)

AI + Google + AI Agent +



Written by Saurabh

33 followers · 6 following

Follow

Senior SWE at Adobe | Python, Go, C++, Distributed Systems | Writing about AI agents & tools I wish someone had explained sooner | fitness, travel, teaching

Responses (1)



Krzysztof

What are your thoughts?

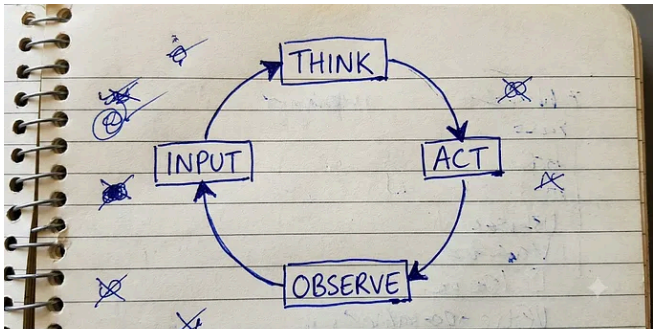
Google Dflix
Oct 2

Built this 6 months ago: converting conversations into atomic notes, structuring knowledge, hybrid search, instant retrieval.

Didn't know Google was standardizing it as OKF 😅 The pattern just makes sense for knowledge that actually compounds. Sometimes the right approach emerges naturally

Reply

More from Saurabh





Saurabh · Sep 13

It's 5AM and I'm Learning How Fable 5.1

Everyone uses these models daily. Almost nobody knows what's actually running...

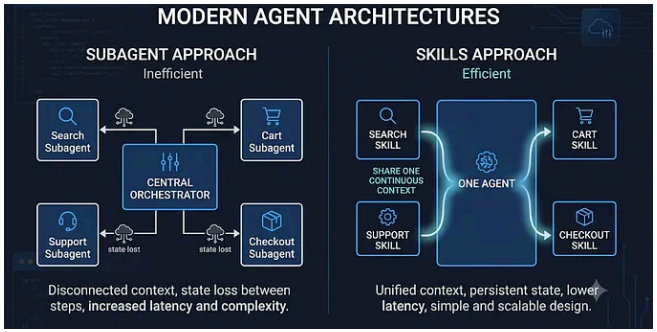
2 1

Saurabh · Sep 26

How I Cracked the Adobe SDE-2 Interview: The Full Strategy, Roun...

The real prep timeline — what I studied, in what order, who I leaned on, and the seven...

1 1

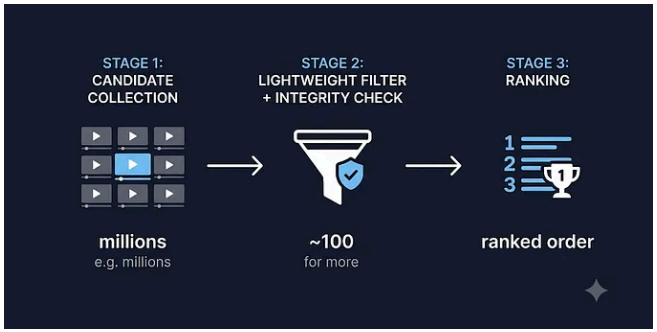


Saurabh · Sep 11

Someone Published the Actual Architecture Real Commerce AI...

Real production data from a year of deploying these systems for actual retailers,...

10



Saurabh · Sep 7

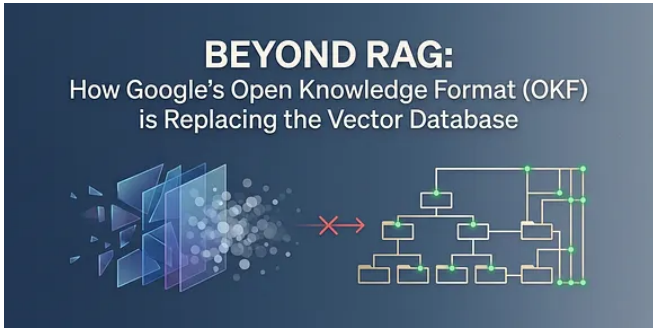
I Posted 20–30 Reels and Never Broke 65K Views.

Every non-tech creator online has "algorithm hacks." I tried them. None of it worked. So I...

4 1

See all from Saurabh

Recommended from Medium

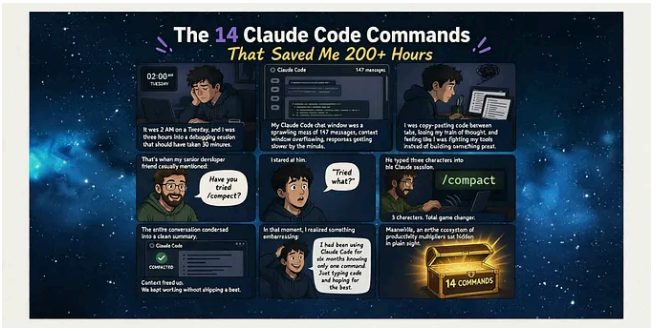


Secret Dev · Jul 3

Beyond RAG: How Google’s Open Knowledge Format (OKF) is...

For the last three years, the default engineering response to any enterprise AI...

3.3K 66 34

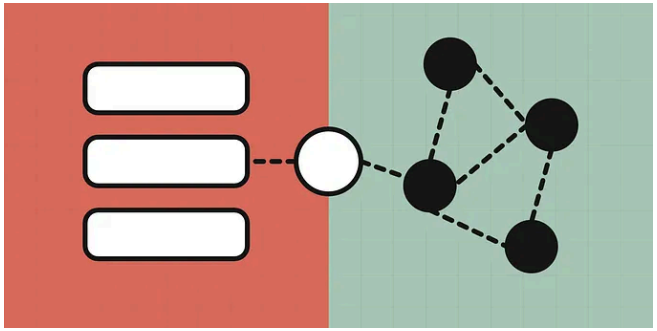


In Towards AI by Mouez Yazidi · Apr 26

I Wasted 6 Months Using Claude Code Wrong. Here Are the 14...

From frustrated beginner to power user: The hidden command ecosystem nobody talks...

6.1K 179 135

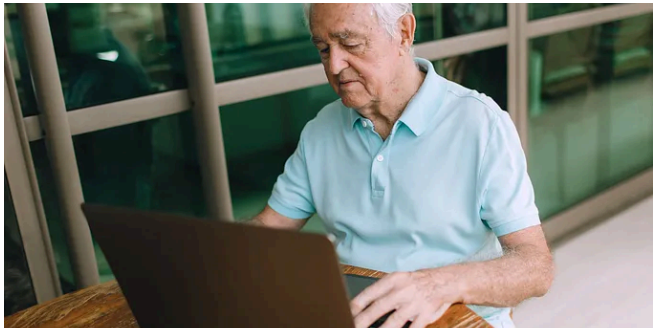


In Data Science Collecti... by Udaykiran Est... · Jul 3

Stop Wasting LLM Tokens: Building a Self-Updating Codebase...

Google's new Open Knowledge Format (OKF) standardizes context. Here's how to build a...

962 26 15

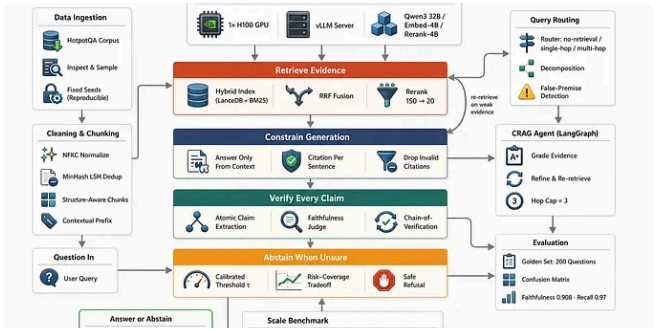


In The Daily Draft by Matt Lillywhite · Jul 7

I'll Instantly Know A Writer Used ChatGPT When I See This

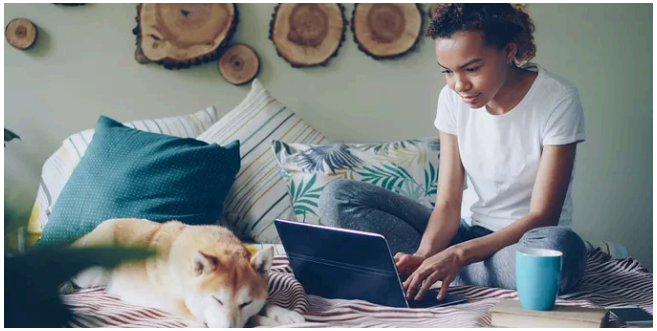
It's pretty damn obvious now that I know what to look for.

32K 1399 571



In Level Up Coding by Fareed Khan · Jun 15

Building a RAG Pipeline for 10M+ Documents With Near-Zero...



In The Sunday Journal by Rafia Naseem · Jul 18

10 Websites Better Than Another Hour of Doomscrolling

Retrieve, constrain, verify, abstain

1.98K 21 45

Here’s to the rabbit holes worth taking!

19K 497 329

See more recommendations